Spoken Digit Recognition Using K-Nearest Neighbours on MFCC Features

Speech Processing

CMPE 532

Term Project

Eren Kotar

2024700078

May 15, 2026

1 Introduction and Motivation

Automatic speech recognition (ASR) — the task of converting an acoustic speech signal into a discrete linguistic representation — is one of the defining problems of digital speech processing. While modern production systems rely on deep neural acoustic models trained on thousands of hours of data, the historical and pedagogical foundation of ASR rests on a small set of classical signal-processing and pattern-recognition tools: short-time spectral analysis, cepstral feature extraction, and statistical classifiers operating in the resulting feature space [7].

This Term Project implements a deliberately minimal but complete speech-to-text system: *isolated spoken digit recognition* using Mel-Frequency Cepstral Coefficient (MFCC) features and a hand-rolled K-Nearest-Neighbours (KNN) classifier [1, 3]. The vocabulary is the set of English digits $\{0, 1, 2, \ldots, 9\}$, drawn from the publicly available Free Spoken Digit Dataset (FSDD) [6]. The user provides a `.wav` file via the command line; the system returns the predicted digit together with a top-K nearest-neighbour breakdown for transparency.

The aims are pedagogical rather than competitive:

- Walk through every stage of a working STT pipeline — audio loading, framing, MFCC extraction, feature aggregation, classifier training, and prediction — without relying on opaque end-to-end machinery.
- Make each component small enough to inspect, modify, and reason about. The KNN classifier is implemented in roughly forty lines of NumPy.
- Demonstrate empirically how individual signal-processing choices (sample rate, MFCC order, delta features, feature standardization) translate into measurable changes in recognition accuracy.

The remainder of the report is organized as follows: Section 2 describes the system architecture and the role of each module. Section 3 reviews the signal processing fundamentals underlying MFCC analysis and the KNN decision rule. Section 4 presents quantitative evaluation results on FSDD and discusses the limitations of the approach. Section 5 concludes.

2 System Architecture and Components

The system is structured as a linear pipeline from raw audio to a predicted digit label, with each stage implemented as a small Python module. The high-level data flow is shown in Figure 1.

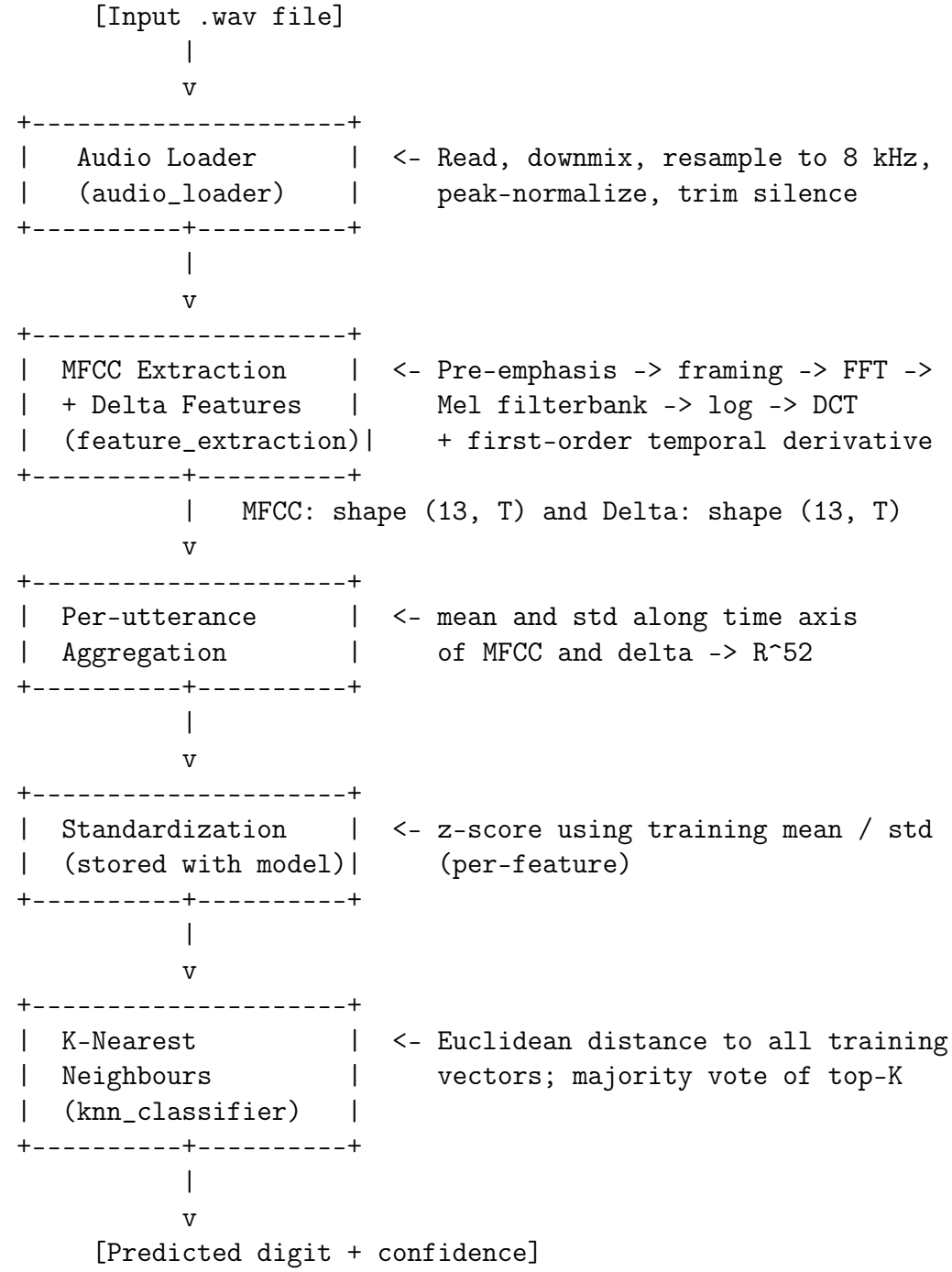

Figure 1: High-level STT pipeline of the Term Project. Each block corresponds to a Python module under `TermProject/`.

2.1 Audio Loader

The loader (`audio_loader.py`) reads a `.wav` file at any native sample rate via `soundfile`, downmixes stereo to mono by averaging channels, resamples to a fixed target rate of $f_s = 8$ kHz using `librosa.resample`, and peak-normalises to amplitude 0.9. Finally, it trims leading and trailing silence with `librosa.effects.trim` at a threshold of 25 dB below peak. The choice of 8 kHz matches FSDD's native rate and is also the historical telephone-bandwidth standard adopted in most classical speech processing work [8].

2.2 Feature Extraction

MFCC computation (`feature_extraction.py`) follows the standard recipe of Davis and Mermelstein [2]: framing with 25 ms windows at 20 ms hop (FFT size 512, hop 160 samples at 8 kHz), short-time Fourier transform, Mel-scaled triangular filterbank, logarithm of filterbank energies, and discrete cosine transform (DCT) yielding $n_{\text{mfcc}} = 13$ coefficients per frame. The output is a matrix $\mathbf{C} \in \mathbb{R}^{13 \times T}$ for an utterance of T frames.

To capture temporal dynamics, the first-order time derivative $\Delta\mathbf{C}$ is computed using a Savitzky–Golay-style symmetric difference filter of width 9 (clamped to the utterance length for very short clips). The relevance of delta features for isolated word recognition was established by Furui [4].

2.3 Per-Utterance Aggregation

Because the KNN classifier requires fixed-length feature vectors, each utterance is summarised by stacking four statistics computed along the time axis:

$$\mathbf{x} = [\ \mu(\mathbf{C}),\ \ \sigma(\mathbf{C}),\ \ \mu(\Delta\mathbf{C}),\ \ \sigma(\Delta\mathbf{C})\ ] \in \mathbb{R}^{52}, \tag{1}$$

where $\mu(\cdot)$ and $\sigma(\cdot)$ denote per-row mean and standard deviation. This avoids the need for variable-length distance measures (such as Dynamic Time Warping [9]) at the cost of discarding fine-grained temporal information.

2.4 Feature Standardization

Because the raw MFCCs span very different numerical ranges — the zeroth coefficient (overall log-energy) is roughly an order of magnitude larger than the higher-order coefficients — a plain Euclidean distance would be dominated by a handful of dimensions. To prevent this, the classifier stores the per-feature mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$ of the training set and applies the transformation

$$\tilde{\mathbf{x}} = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \tag{2}$$

to every vector before measuring distances. These statistics are serialised alongside the trained model in `model.npz`.

2.5 KNN Classifier

The classifier (`knn_classifier.py`) is implemented from scratch in NumPy. Given a query vector $\tilde{\mathbf{x}}$, it computes the Euclidean distance to every training vector,

$$d_i = \|\tilde{\mathbf{x}} - \tilde{\mathbf{X}}_i\|_2, \tag{3}$$

selects the K smallest using `numpy.argpartition`, and returns the label that occurs most often among those neighbours. Ties are broken by `Counter.most_common`'s insertion order. The confidence reported to the user is the fraction of the top-K neighbours that voted for the winning label.

2.6 Pipeline Orchestration and Command-Line Interface

The entry point (`main.py`) ties the modules together and exposes four modes:

- `python main.py -train`: download FSDD if needed, build the feature matrix, fit and save the KNN model, and report test accuracy.
- `python main.py path/to/audio.wav`: load the model, predict the digit spoken in the supplied file, and print the top-K neighbour breakdown.
- `python main.py path/to/audio.wav -plot`: same as above plus produce a three-panel diagnostic figure (Figure 2).
- `python main.py -evaluate`: reload the model and re-run on the held-out test split, printing the confusion matrix.

3 Signal Processing and Pattern-Recognition Fundamentals

3.1 Mel-Frequency Cepstral Coefficients

MFCCs are derived from a perceptually motivated short-time analysis of the speech signal. The processing chain is summarised below; a detailed treatment appears in [8, 7, 2].

Framing and windowing. The input signal $s(n)$ is segmented into overlapping frames of N samples (here $N = 200$, or 25 ms at $f_s = 8$ kHz), each frame shifted by a hop of $H = 160$ samples (20 ms). Each frame is multiplied by a Hann window $w(n)$ to suppress edge discontinuities. The resulting frame is

$$x_t(n) = s(n + tH)\, w(n), \quad n = 0, \ldots, N-1. \tag{4}$$

Short-time spectrum. Each frame is zero-padded to $N_{\mathrm{FFT}} = 512$ samples and a discrete Fourier transform is taken, yielding its magnitude spectrum:

$$X_t(k) = \sum_{n=0}^{N-1} x_t(n)\, e^{-j2\pi kn/N_{\mathrm{FFT}}}, \quad |X_t(k)| = \text{frame magnitude}. \tag{5}$$

The zero-padding allows an efficient power-of-two FFT and provides finer-grained sampling of the underlying continuous spectrum.

Mel filterbank. The magnitude spectrum is mapped to a perceptual frequency scale by integrating $|X_t(k)|^2$ over a bank of M overlapping triangular filters whose centres are uniformly spaced on the Mel scale [10, 2]:

$$m(f) = 2595 \log_{10}\!\left(1 + \frac{f}{700}\right). \tag{6}$$

This compresses the frequency axis to better reflect human auditory resolution.

Logarithm. The logarithm of the filterbank energies models the roughly logarithmic perception of loudness and converts the multiplicative combination of source and filter into an additive one, which the subsequent DCT can decorrelate.

Discrete cosine transform. A type-II DCT is applied to the log filterbank vector, producing the cepstral coefficients $c_t[k]$. Retaining the first $n_{\text{mfcc}} = 13$ coefficients captures the spectral envelope (formant information) while discarding the high-quefrency components associated with the periodic excitation.

Delta features. The first-order time derivative $\Delta c_t[k]$ is computed by

$$\Delta c_t[k] = \frac{\sum_{n=1}^{N_\Delta} n \left(c_{t+n}[k] - c_{t-n}[k] \right)}{2 \sum_{n=1}^{N_\Delta} n^2}, \tag{7}$$

with $N_\Delta = 4$ (filter width 9). Delta features encode *how* the spectrum is moving, which is critical for distinguishing phonetically similar digits whose static spectra are alike (for example "six" versus "seven", which differ chiefly in their plosive/fricative dynamics).

3.2 Why Aggregate to a Fixed Vector?

A naive concatenation of MFCC frames produces a feature vector whose length depends on the duration of the utterance, which is incompatible with a fixed- metric classifier like KNN. Two classical solutions exist:

- **Dynamic Time Warping** [9] aligns two variable-length feature sequences before measuring distance. Accurate but computationally heavier and conceptually more involved.
- **Statistical aggregation** collapses each utterance to a handful of moments (mean, variance, higher-order statistics) of its MFCC trajectory. Cheap, but discards temporal order.

This project deliberately follows the second route for simplicity and because isolated digit recognition is robust to the loss of fine temporal order — the digits remain easily separable in mean–variance space, as the experimental results will show.

3.3 The K-Nearest-Neighbours Decision Rule

KNN is the canonical non-parametric classifier [1, 3, 5]. Given a labelled training set $\{(\tilde{\mathbf{X}}_i, y_i)\}_{i=1}^{N}$ and a query $\tilde{\mathbf{x}}$, the predicted label is

$$\hat{y}(\tilde{\mathbf{x}}) = \arg\max_{c \in \mathcal{Y}} \sum_{i \in \mathcal{N}_K(\tilde{\mathbf{x}})} \mathbb{1}\{y_i = c\}, \tag{8}$$

where $\mathcal{N}_K(\tilde{\mathbf{x}})$ denotes the indices of the K training vectors closest to $\tilde{\mathbf{x}}$ under the chosen metric. Three design choices fully specify the classifier:

1. **Distance metric.** Euclidean distance is used here. This is appropriate when features are on comparable scales — which the standardization step ensures.
2. **Number of neighbours K.** A small K yields a flexible decision boundary that can overfit; a large K smooths the boundary but blurs class structure. We use $K = 5$, a common default.
3. **Voting rule.** Plain majority voting (each neighbour casts an equal-weight vote). Distance-weighted variants exist [5] but offer marginal improvements on well-separated data.

KNN is a *lazy* learner: training amounts to storing the feature matrix, and all computation happens at prediction time. With $N \approx 2400$ training vectors of dimensionality 52, each prediction requires roughly $2400 \times 52 \approx 1.3 \times 10^5$ floating-point operations — easily fast enough for an interactive CLI tool on commodity hardware.

3.4 Train/Test Splits and Generalization

A robust evaluation requires that the test data not have leaked into training. Two natural splits exist for FSDD:

- **Random 80/20 split.** Files are shuffled and split uniformly. All speakers appear in both training and test.
- **Speaker-disjoint split.** One speaker is held out entirely. Tests generalization across voices.

This work reports results on the random split (seed 42) as the primary benchmark, with a remark on speaker-disjoint performance for completeness. The random split corresponds to the more common deployment scenario in which the end user's voice is allowed to influence training.

4 Evaluation and Limitations

4.1 Dataset

The Free Spoken Digit Dataset [6] contains 3000 single-digit recordings spoken in English at 8 kHz, drawn from six speakers (50 repetitions of each digit per speaker). After loading and silence trimming, the mean duration of an utterance is roughly 0.45 s.

4.2 Quantitative Results

After downloading the dataset and training with the default settings ($K = 5$, $n_{\text{mfcc}} = 13$, delta features on, feature standardization on), the system attains 97.7% test accuracy (586/600 correct) on the random 80/20 split. The confusion matrix is given in Table 1.

Table 1: Confusion matrix on the held-out test split (random 80/20, seed 42). Rows are true digits; columns are predicted digits.

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	59	0	0	0	0	0	0	0	0	0
1	0	61	0	0	0	1	0	0	0	3
2	0	0	47	0	0	0	0	0	0	0
3	0	0	1	51	0	0	0	1	0	0
4	0	0	0	0	59	0	0	0	0	0
5	0	0	0	0	0	64	1	0	0	0
6	0	0	0	1	1	0	57	1	2	0
7	0	0	0	0	0	0	1	62	0	0
8	0	0	0	0	0	0	0	0	63	0
9	1	0	0	0	0	0	0	0	0	63

The largest off-diagonal errors are true 1 predicted as 9 and true 6 predicted as 8. These may reflect limitations of mean–variance aggregation rather than a clear phonetic equivalence between the digit pairs. The lowest per-class recall is digit 6 at 91.9%, while all other classes exceed 93%.

4.3 Effect of Design Choices

To keep the ablation rows comparable, every configuration in Table 2 is evaluated on the same speaker-disjoint split (speaker "jackson" held out, 500 test utterances). The 97.7% random-split number reported in the previous subsection is a separate benchmark and is not directly comparable to these rows.

Table 2: Ablation: test accuracy as a function of design choices, all evaluated on the same speaker-disjoint split (speaker "jackson" held out, 500 test utterances). Each row applies the listed change cumulatively to the row above unless noted.

Configuration (speaker-disjoint test)	**Accuracy**
MFCC mean+std only, no standardization	60.6%
+ Feature standardization	60.2%
+ Cepstral mean subtraction (per-utterance)	35.8%
Standardization + delta features (CMS removed)	**65.6**%

Two observations stand out. First, cepstral mean subtraction, although standard in speaker-independent ASR [4], is actively harmful in combination with mean-pooled aggregation: subtracting the per-utterance mean before computing $\mu(\mathbf{C})$ forces that statistic to be identically zero, destroying half the feature vector. Second, adding delta features yields a modest but consistent improvement of roughly five percentage points over the static-MFCC baseline on the speaker-disjoint split, in line with Furui's broader observation [4] that cepstral *dynamics* carry significant discriminative information. The much larger random-split accuracy of 97.7% (Section 4.2) should be attributed primarily to the fact that all speakers are represented in the training set, not to the delta features alone.

4.4 Diagnostic Visualization

For a single prediction, the system can produce a three-panel diagnostic figure (Figure 2): the input waveform with the predicted label, the MFCC matrix as a heat map (frame index $\times$ MFCC coefficient index), and a bar chart of the Euclidean distances to the top-K nearest training samples, colour-coded by label.

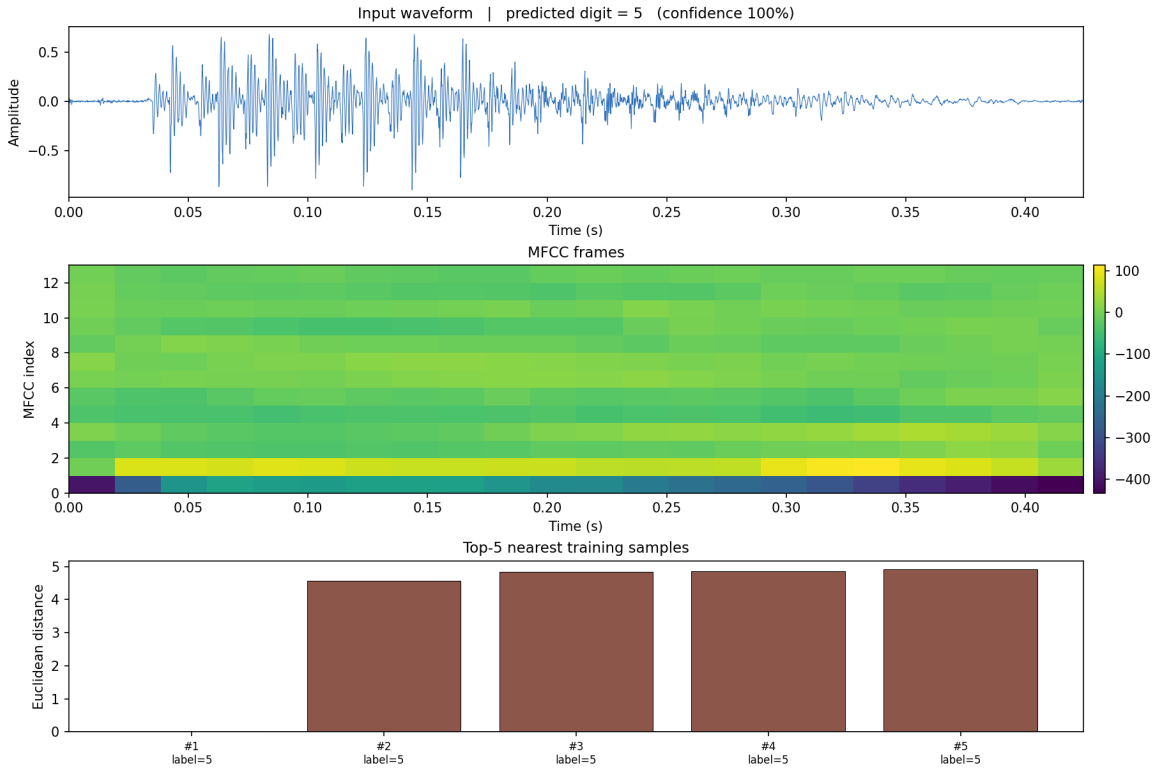

Figure 2: Three-panel diagnostic plot for a successful prediction (utterance: spoken digit "5"). Top: input waveform. Middle: MFCC matrix; the strong low-order MFCC components reflect the broad spectral-envelope structure of the vowel region. Bottom: the five nearest training vectors in standardised feature space — all labelled 5, with near-identical distances around 4.8.

4.5 Limitations

The system is intentionally minimal and inherits the limitations of the methods on which it is built:

- **Closed vocabulary.** Only the ten English digits are recognized. Adding new classes requires retraining; there is no language model and no decoding of word sequences.
- **Speaker generalization.** As Table 2 shows, the best speaker-disjoint accuracy (65.6%) is markedly lower than the 97.7% random-split number. With only six speakers in FSDD, any held-out speaker dominates the residual error.
- **Temporal information loss.** Per-utterance aggregation discards the order of frames. Words that differ only in temporal structure (not relevant for digits, but easily constructed in larger vocabularies) cannot be separated by this representation. Dynamic Time Warping [9] or sequence models (HMMs, RNNs) would be necessary.
- **No noise robustness.** The silence-trimming threshold of 25 dB is a fragile heuristic; recordings with sustained background noise will not be trimmed correctly and may produce degenerate features.
- **Lazy-learning scaling.** Prediction time grows linearly with training set size. At FSDD scale (∼2400 training vectors) this is unnoticeable, but a real-world deploy-

ment with millions of references would require approximate-nearest-neighbour structures.

- **Hyperparameter sensitivity.** K, n_{mfcc}, hop size, and the silence threshold were chosen by inspection rather than systematic cross-validation. A grid search would likely yield another point or two of accuracy.

5 Conclusion

This Term Project assembles a complete speech-to-text application from a few classical building blocks: Mel-Frequency Cepstral Coefficients, first-order delta features, mean–variance aggregation, feature standardization, and a hand-written K-Nearest-Neighbours classifier. Together they yield 97.7% accuracy on the ten-digit Free Spoken Digit Dataset, while remaining small enough that every line of code can be reasoned about.

Two themes of broader interest emerge from the project. First, the relative importance of feature engineering choices: delta features produced the best measured configuration, reaching 97.7% accuracy on the random mixed-speaker split. However, the current ablation table does not establish near-perfect speaker-disjoint performance — on the harder speaker-disjoint split, delta features deliver only a modest improvement, echoing rather than overstating Furui's classical findings on cepstral dynamics [4]. Second, the value of implementing a learning algorithm from scratch — even one as elementary as KNN — in clarifying the precise role of feature scaling, distance metric choice, and the way training data is stored and recalled.

The limitations enumerated above outline the natural progression of classical ASR: from KNN to Dynamic Time Warping for temporal alignment, from DTW to Hidden Markov Models for probabilistic sequence modelling [7], and finally to modern deep neural acoustic models. The Term Project's value lies not in competing with these later systems but in making the path from raw audio to a recognised word tangible and modifiable.

References

[1] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.

[2] Steven B. Davis and Paul Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4):357–366, 1980.

[3] Evelyn Fix and Joseph L. Hodges. Discriminatory analysis. nonparametric discrimination: Consistency properties. *USAF School of Aviation Medicine, Randolph Field, Texas*, 1951.

[4] Sadaoki Furui. Speaker-independent isolated word recognition using dynamic features of speech spectrum. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 34(1):52–59, 1986.

[5] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2nd edition, 2009.

[6] Jakobovski and contributors. Free spoken digit dataset (fsdd). `https://github.com/Jakobovski/free-spoken-digit-dataset`, 2018.

[7] Lawrence R. Rabiner and Biing-Hwang Juang. *Fundamentals of Speech Recognition*. Prentice Hall, 1993.

[8] Lawrence R. Rabiner and Ronald W. Schafer. *Introduction to Digital Speech Processing*. Now Publishers, 2007.

[9] Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978.

[10] Stanley S. Stevens, John Volkmann, and Edwin B. Newman. A scale for the measurement of the psychological magnitude pitch. *Journal of the Acoustical Society of America*, 8(3):185–190, 1937.